Python Handbook

A working book · Scratch → Advanced

FRONT MATTER

Title Page

The Learning Path

BEGINNER · MODULES

1 · Getting Started 01

2 · Variables & Types 02

3 · Numbers & Math 03

4 · Strings 04

5 · Booleans & Operators 05

6 · Lists 06

7 · Tuples 07

8 · Dictionaries 08

9 · Sets 09

10 · Control Flow 10

11 · Loops 11

12 · Comprehensions 12

INTERMEDIATE (soon)

Functions 13

Modules & Packages 14

Errors & Exceptions 15

Files & I/O 16

Classes & OOP 17

Inheritance & Dunders 18

ADVANCED (soon)

Iterators & Generators 19
Decorators & Closures 20
Type Hints & Dataclasses 21
Standard Library 22
Concurrency 23
Testing & Packaging 24

A WORKING BOOK · FIRST EDITION · 2026

The Python Handbook

Learn Python from your first line of code to professional, idiomatic programs. Every topic is explained, defined, demonstrated, stress-tested against its edge cases, and practised.

Beginner through Advanced

Concepts · Definitions · Worked Examples · Edge Cases · Things to Think About · Exercises with Solutions

The Learning Path

Twelve stages across three tiers. Walk it in order if you are new; open any module directly if you are not. Each stage names what you will be able to do once you finish it.

BEGINNER — *you can write working programs*

Stage 1

First Steps

1 · Getting Started

Run code, print, read input.

Stage 2

Data Types

2 · Variables & Types

3 · Numbers & Math

4 · Strings

5 · Booleans

Store and combine every basic value.

Stage 3

Collections

6 · Lists

7 · Tuples

8 · Dictionaries

9 · Sets

Group and look up many values.

Stage 4

Control Flow

10 · If / Elif / Else

11 · Loops

12 · Comprehensions

Decide and repeat.

INTERMEDIATE

you can structure real programs

Stage 5

Functions

13 · Functions & Scope

Reusable, named logic.

Stage 6

Organization

14 · Modules & Packages

Many files, real libraries.

Stage 7

Robustness

15 · Errors

16 · Files & I/O

Handle failure and the outside world.

Stage 8

Objects

17 · Classes & OOP

18 · Inheritance

Model things with behaviour.

ADVANCED

you write professional, idiomatic Python

Stage 9

Pythonic Power

19 · Generators

20 · Decorators

Lazy, composable, elegant code.

Stage 10

Modern Python

21 · Type Hints

Safety and clarity.

Stage 11

The Library

22 · Standard Library

Batteries, not reinvention.

Stage 12

Professional

23 · Concurrency

24 · Testing

Correct, tested, parallel.

MODULE 1 · BEGINNER · STAGE 1

Getting Started

How Python runs your code, and the two commands every program is built from: showing output and reading input.

Needs: nothing **Teaches:** running code · print · input · comments **Then you can:** write and run small programs

CONCEPT What "running Python" means

Python is an **interpreted** language. You do not translate your program into the processor's native instructions yourself; you write plain text in a file ending in `.py`, hand it to a program called the interpreter, and it carries out your instructions one line at a time, top to bottom. There is no separate build step, so the cycle of write, run, look at the result, and adjust is fast. That speed is a real part of why Python is a comfortable first language.

You can feed the interpreter code in two ways. Save it in a file and run the whole file at once, which is how real programs are written and shared. Or start the interpreter in **interactive mode**,

called the REPL, and type one line at a time, seeing each result instantly, which is perfect for trying an idea. A program talks to the world through two basic actions: `print()` sends text out to the screen, and `input()` reads a line the user types in.

DEFINITION **Interpreter**

The program named `python` that reads your `.py` file and runs it line by line. You never compile Python into a standalone app; the interpreter is always what runs it.

EXAMPLES Code you can run

Example 1.1 · Hello, World and reading a name

PYTHON

```python
# save as hello.py, then run:  python hello.py
print("Hello, World!")              # send a line of text to the screen
name = input("What is your name? ")    # show a prompt, wait for the user, store t
print(f"Nice to meet you, {name}.")  # f-string drops the value of name into the t
```

OUTPUT

```
Hello, World!
What is your name? Ada
Nice to meet you, Ada.
```

Read it top to bottom, exactly as the interpreter does. Line 2 prints. Line 3 pauses for input. Line 4 prints again, using the value collected on line 3.

Example 1.2 · Controlling how `print` behaves

PYTHON

```python
print("a", "b", "c")           # several values, separated by spaces by default
print("a", "b", sep="-")      # sep= changes what goes BETWEEN the values
print("loading", end="...")  # end= changes what goes AFTER (default is a newline)
print("done")                  # so this lands on the SAME line as 'loading'
```

OUTPUT

```
a b c
a-b
loading...done
```

By default `print` joins items with a space and ends with a newline. The `sep` and `end` options let you change both, which matters the moment you want output on one line.

EDGE CASES What goes wrong, and the error you will see

⚠ EDGE CASES & COMMON ERRORS

Run these on purpose once, so the error messages become familiar rather than frightening.

PYTHON

```python
print("hi"            # forgot the closing ) → SyntaxError
# SyntaxError: '(' was never closed

  print("oops")        # a stray leading space → IndentationError
# IndentationError: unexpected indent

print(naem)            # a typo'd name that was never defined → NameError
# NameError: name 'naem' is not defined
```

Three of the four errors you will meet in your first hour are here: a missing bracket, an accidental indent, and a misspelled name. The interpreter always names the error type and the line, so read the last line of a traceback first.

? THINK ABOUT IT

Before you read on, predict what each of these prints, and why:

1. `print("2" + "3")` versus `print(2 + 3)`
2. `print("ha" * 3)`

▸ Reveal the thinking

REFERENCE Quick reference

Goal	Code
Show text	`print("hello")`
Show several values	`print(a, b, c)`
Custom separator / ending	`print(a, b, sep=", ", end="!\n")`

Read text from the user	`name = input("Prompt: ")`
Read a number	`n = int(input())`
A comment (ignored by Python)	`# a note to a human`

✎ EXERCISES

1. Print `Hello, World!`.
2. Ask for the user's city and print `You live in <city>.`
3. Print `1`, `2`, `3` on one line separated by commas.
4. Make `Loading` and `Done` appear on the same line.
5. **Think:** what is the difference between `print(3 * 3)` and `print("3" * 3)`? Predict, then test.

▸ Show answers

MODULE 2 · BEGINNER · STAGE 2

Variables & Data Types

How a program remembers things, why a name is a label rather than a box, and the small set of types every value belongs to.

Needs: Module 1 **Teaches:** assignment · types · conversion · naming **Then you can:** store any kind of value

CONCEPT Names, values, and types

A running program constantly needs to remember things: a user's name, a running total, a result it will reuse later. A **variable** gives a value a name so you can refer back to it. You

create one with the equals sign, name on the left, value on the right, and from then on the name stands for that value.

Two ideas set Python apart. First, you never declare what *kind* of value a name will hold. The value carries its own type, and the same name can hold a number now and a string later. This is **dynamic typing**. Second, and more subtly, a variable is not a box that contains a value; it is a **label tied to a value**. When you write `x = 5`, the number 5 exists somewhere in memory and `x` is a tag pointing at it. Writing `x = 7` does not alter the 5; it moves the tag to a different object. This picture matters because two labels can point at the very same object, which you will meet as a real bug in the Lists module.

Every value belongs to a **type**, which decides what it can do. The handful below cover nearly everything. A value of one type can often be converted to another, for instance turning the text `"42"` into the number `42`, which you will do constantly because user input always arrives as text.

> DEFINITION **Variable**
>
> A name bound to a value. The binding is made by assignment (`name = value`) and can change at any time. The name does not contain the value; it refers to a value object in memory, and several names may refer to the same one.

> DEFINITION **Data Type**
>
> The category a value belongs to (`int`, `str`, `list`, and so on), which decides what operations it supports. Check it with `type(value)`.

EXAMPLES Code you can run

Example 2.1 · Assigning, and inspecting the type

PYTHON

```python
age = 28              # an int (whole number)
price = 19.99         # a float (has a decimal point)
city = "London"       # a str (text, in quotes)
print(type(age), type(price), type(city))   # ask each value its type
```

OUTPUT

```
<class 'int'> <class 'float'> <class 'str'>
```

We never wrote the types. Python read them from the values themselves. The quotes are what make `city` text rather than a name.

Example 2.2 · Converting text into a number

```python
reply = input("How many? ")   # input ALWAYS returns text, even if it looks numeri
count = int(reply)            # convert the text "4" into the number 4
print(count * 3)               # now multiplication does arithmetic
```

```
OUTPUT
How many? 4
12
```

Drop the `int()` and `count * 3` would give `'444'`, because `count` would still be the string `"4"` and `*` repeats text. Converting at the boundary is the fix.

EDGE CASES What goes wrong, and the error you will see

⚠ EDGE CASES & COMMON ERRORS

```python
int("3.5")        # int() only parses whole numbers, not decimals
# ValueError: invalid literal for int() with base 10: '3.5'

int("twelve")     # words are not digits
# ValueError: invalid literal for int() with base 10: 'twelve'

"5" + 3           # you cannot add a string and a number
# TypeError: can only concatenate str (not "int") to str

age               # using a name you never assigned
# NameError: name 'age' is not defined
```

Notice the two distinct error *types*. A **ValueError** means the value is the wrong shape (text that is not a number). A **TypeError** means the types do not fit the operation (string plus int). Learning to read which one you got tells you whether to fix the value or fix the operation.

? THINK ABOUT IT

You write `x = "5"`. Then you want to add 3 to it. There are two ways to make `8` happen and one way to make `"53"` happen. Work out all three before revealing.

▸ Reveal the thinking

REFERENCE Quick reference

Type	Example	Holds
`int`	`42`	whole numbers, any size
`float`	`3.14`	decimals (approximate)
`str`	`"hi"`	text
`bool`	`True`	truth values
`list`	`[1, 2]`	an ordered, changeable group
`dict`	`{"a": 1}`	key-to-value lookups
`NoneType`	`None`	the deliberate absence of a value

Task	Code
Check a type	`type(x)` · `isinstance(x, int)`
Text → number	`int("42")` · `float("3.14")`
Number → text	`str(42)`
Swap two values	`a, b = b, a`

TIP Use `=` to assign and `==` to compare. `if x = 5` is an error; you want `if x == 5`. This slip catches everyone once.

✎ EXERCISES

1. Create `name`, `price`, and `in_stock` (a boolean) and print all three.
2. Swap `a = 1` and `b = 2` in one line.

3. Read two numbers and print their sum (remember the conversion).
4. **Edge case:** what error does `int("3.5")` raise, and how would you convert `"3.5"` to the number 3 instead?
5. **Think:** after `x = 10; y = x; x = 20`, what is `y`? Explain using labels and values.

▸ **Show answers**

MODULE 3 · BEGINNER · STAGE 2

Numbers & Math

Two number types, seven operators, the two divisions that confuse everyone, and why `0.1 + 0.2` is not `0.3`.

Needs: Module 2 **Teaches:** int · float · operators · math module **Then you can:** do any calculation

CONCEPT Integers, floats, and the operators

Python has two everyday number types. An **int** is a whole number, and remarkably it has no size limit, so `2 ** 1000` gives an exact answer with three hundred digits. A **float** is a number with a decimal point, used for measurements and fractions. The two mix freely, and the result rises to the more general type, so an `int` plus a `float` is a `float`.

The operators are mostly familiar, with two that deserve attention. Division with `/` always gives a float, even when it divides evenly, so `6 / 2` is `3.0`. When you want a whole-number quotient use **floor division** `//`, and its partner **modulo** `%` gives the remainder. Those two answer questions like "is this number even?" and "how many full hours are in these minutes?". For anything beyond arithmetic, the `math` module supplies roots, rounding, and constants.

DEFINITION

Integer and Float

An `int` is a whole number of unlimited size. A `float` is a number with a fractional part, stored in fixed memory and therefore *approximate*. Use `int` for counting and indexing, `float` for measurement.

EXAMPLES Code you can run

Example 3.1 · The seven operators

PYTHON

```python
print(7 + 2)    # 9    add
print(7 - 2)    # 5    subtract
print(7 * 2)    # 14   multiply
print(7 / 2)    # 3.5  TRUE division → always a float
print(7 // 2)   # 3    FLOOR division → drops the fraction
print(7 % 2)    # 1    MODULO → the remainder
print(7 ** 2)   # 49   power
```

OUTPUT

```
9
5
14
3.5
3
1
49
```

The two to memorise are `/` (always a float) and `//` with `%` (whole quotient and remainder). Mixing them up is the single most common numeric slip.

Example 3.2 · Even-or-odd, and the math module

PYTHON

```python
n = 17
print(n % 2 == 0)        # False → 17 leaves remainder 1, so it is odd

import math                  # bring in the math toolkit
area = math.pi * 5 ** 2      # pi × radius squared
print(round(area, 2))        # round to 2 decimal places
```

OUTPUT

```
False
78.54
```

The `x % 2 == 0` test for evenness is everywhere in programming. `import math` unlocks `pi`, `sqrt`, and dozens more.

EDGE CASES

What goes wrong, and the surprises

⚠ EDGE CASES & COMMON ERRORS

PYTHON

```python
10 / 0            # dividing by zero is undefined
# ZeroDivisionError: division by zero

0.1 + 0.2         # floats are approximate, so this is NOT 0.3
# 0.30000000000000004

round(2.5)          # surprise: rounds to the nearest EVEN number
# 2     (not 3, this is 'banker's rounding')

10 + "5"          # number plus text never works
# TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

The float result is not a bug; it is how all computers store decimals. Compare floats with `math.isclose(a, b)` rather than `==`, and for money use `decimal.Decimal("0.1")`. The `round(2.5) == 2` surprise trips up nearly everyone the first time.

? THINK ABOUT IT

You have `total_minutes = 137`. Without looking below, how would you get "2 hours and 17 minutes" using only `//` and `%`? Then ask: why does `137 // 60` give the hours and `137 % 60` give the minutes?

▸ Reveal the thinking

REFERENCE

Quick reference

Call	Result	Use

`abs(-5)`	`5`	distance from zero
`round(3.14159, 2)`	`3.14`	round to n places
`min(3,1,2)` · `max(...)`	`1` · `3`	smallest / largest
`sum([1,2,3])`	`6`	total a list
`divmod(17, 5)`	`(3, 2)`	quotient and remainder at once
`math.sqrt(144)`	`12.0`	square root
`math.isclose(a, b)`	`True/False`	safe float comparison

✎ EXERCISES

1. Compute the area of a circle of radius 7, rounded to one decimal.
2. Use `%` to test whether 100 is divisible by 7.
3. Use `divmod` to split 130 minutes into hours and minutes.
4. **Edge case:** what does `10 / 0` raise, and what does `10 // 0` raise?
5. **Think:** show `0.1 + 0.2 == 0.3` is `False`, then make a correct comparison and explain why the first failed.

▸ Show answers

MODULE 4 · BEGINNER · STAGE 2

Strings

Text as an ordered sequence: how to index it, slice it, format it with f-strings, and why you can never change one in place.

Needs: Module 2 | **Teaches:** indexing · slicing · methods · f-strings | **Then you can:** manipulate any text

CONCEPT An ordered, unchangeable sequence of characters

A **string** is a piece of text written inside quotes, single or double, it makes no difference. It is an ordered **sequence** of characters, which means every character has a position, called an index, starting at 0, and you can pull out one character or a whole range. Because text is everywhere in real programs, strings carry a rich set of built-in methods for searching, cleaning, splitting, and reassembling.

One property shapes everything else: strings are **immutable**. You cannot change a character inside an existing string. Every operation that seems to modify a string actually builds and returns a brand new one, leaving the original untouched. So `s.upper()` hands you a new uppercase string; if you want to keep it you must capture it with `s = s.upper()`. The modern way to build text from values is the **f-string**: put an `f` before the opening quote and write expressions inside braces.

> **DEFINITION** **String**
>
> An immutable, ordered sequence of characters, written in quotes. Immutable means it can never change in place; operations that transform it return a new string. Reach characters by index (`s[0]`) and ranges by slice (`s[1:4]`).

EXAMPLES Code you can run

Example 4.1 · Indexing and slicing

PYTHON

```python
s = "Python"
print(s[0])     # 'P'      index 0 is the FIRST character
print(s[-1])    # 'n'      negative counts from the end
print(s[0:3])   # 'Pyt'    from 0 up to (NOT including) 3
print(s[2:])    # 'thon'   leave the end off → to the end
print(s[::-1])  # 'nohtyP' step of -1 → reversed
```

OUTPUT

```
P
n
Pyt
thon
nohtyP
```

The slice `[start:stop]` stops *before* `stop`, which is why `s[0:3]` gives three characters, not four. The reversal trick `[::-1]` is worth memorising.

Example 4.2 · f-strings, splitting, and joining

PYTHON

```python
name, score = "Ada", 91.5
print(f"{name} scored {score:.1f}")  # :.1f → one decimal place
print(f"{1234567:,}")               # :, → thousands separators

parts = "ada,bob,carol".split(",")   # text → list, breaking on commas
print(parts)
print(" & ".join(parts))            # list → text; join is called on the SEPARATOR
```

OUTPUT

```
Ada scored 91.5
1,234,567
['ada', 'bob', 'carol']
ada & bob & carol
```

`split` and `join` are inverses and do a huge share of real text work. The quirk to remember: you call `join` on the separator string, handing it the list.

EDGE CASES

What goes wrong, and the error you will see

⚠ EDGE CASES & COMMON ERRORS

PYTHON

```python
s = "Python"
s[0] = "J"          # strings are immutable → you cannot assign to a character
# TypeError: 'str' object does not support item assignment

s[10]              # reaching past the end
# IndexError: string index out of range
```

```
s.upper()          # this does NOT change s; the new string is thrown away
print(s)           # still 'Python', you forgot to capture the result
# Python
```

The silent trap is the last one: `s.upper()` looks like it works but changes nothing, because strings are immutable and the new string was never stored. Write `s = s.upper()`. The first two raise loud errors; this one fails quietly, which is worse.

? THINK ABOUT IT

You want to capitalise the first letter of `"hello"` to get `"Hello"`, but `s[0] = "H"` raises an error. Given that strings are immutable, how can you build `"Hello"` instead?

▸ Reveal the thinking

REFERENCE Quick reference

Method	Example → result
`.upper() .lower()`	`"Hi".upper()` → `'HI'`
`.strip()`	`" hi ".strip()` → `'hi'`
`.replace(a, b)`	`"cat".replace("c","b")` → `'bat'`
`.split(sep)`	`"a,b".split(",")` → `['a','b']`
`sep.join(list)`	`",".join(['a','b'])` → `'a,b'`
`.find(x) .count(x)`	position of · how many
`.startswith() .endswith()`	`"a.csv".endswith(".csv")` → `True`
`x in s`	`"Py" in "Python"` → `True`

✎ EXERCISES

1. Print the first and last character of `"handbook"`.
2. Reverse `"stressed"` with a slice (you get a real word).
3. Count the vowels in `"programming"`.
4. **Edge case:** why does `"abc"[0] = "z"` fail, and how do you produce `"zbc"` instead?

5. **Think:** `s = " Hi "`. Predict the result of `s.strip().lower()` and explain the order the two methods run in.

▸ **Show answers**

MODULE 5 · BEGINNER · STAGE 2

Booleans & Operators

True and False, the comparisons that produce them, the logic that combines them, and the surprising rule that every value is already either true or false.

Needs: Module 2 **Teaches:** comparison · logic · truthiness **Then you can:** express any condition

CONCEPT Truth, comparison, and truthiness

A **boolean** is one of two values, `True` or `False`. They are the answers to yes-or-no questions, and every decision a program makes rests on one. You rarely type them directly; you produce them by **comparing** values with `==` (equal), `!=` (not equal), and `<`, `>`, `<=`, `>=`. To build bigger conditions you combine booleans with `and`, `or`, and `not`.

Here is the idea that surprises newcomers and quickly becomes indispensable: **every** value can act as true or false, not just real booleans. Zero, the empty string, and empty containers count as false; everything else counts as true. This is **truthiness**, and it lets you write `if name:` to mean "if the name is not empty". A second surprise follows from it: `and` and `or` do not return `True`/`False`, they return one of the actual operands, which is what makes `x = value or default` a clean way to supply a fallback.

DEFINITION Boolean & Truthiness

A boolean is `True` or `False`. Truthiness is the rule that any value can stand in for a boolean in a condition: `0`, `0.0`, `""`, `[]`, `{}`, and `None` are "falsy"; everything else is "truthy".

EXAMPLES Code you can run

Example 5.1 · Comparing and combining

```python
age = 20
has_ticket = True
print(age >= 18 and has_ticket)   # and → BOTH must be true
print(age < 13 or age > 65)      # or → EITHER being true is enough
print(1 <= age <= 30)            # chained: reads "age is between 1 and 30"
```

```
OUTPUT
True
False
True
```

Read the operators aloud and they say what they mean. The chained comparison `1 <= age <= 30` is a Python convenience most languages lack.

Example 5.2 · Truthiness and the "or" default

```python
username = ""                  # an empty string is FALSY
display = username or "Guest"  # or returns the first truthy value
print(display)                  # 'Guest', because "" was falsy
print(bool(""), bool("hi"), bool([]), bool([0]))
```

```
OUTPUT
Guest
False True False True
```

Note the last value: `[0]` is truthy. A container is truthy when it is *non-empty*, no matter what is inside it. The single zero does not make the list falsy.

EDGE CASES The traps that produce wrong answers

⚠ EDGE CASES & COMMON ERRORS

```python
if x = 5:            # using = (assign) where you meant == (compare)
# SyntaxError: invalid syntax. Maybe you meant '==' ?

bool("False")        # the TEXT "False" is a non-empty string → truthy!
# True   (this surprises everyone)

bool(" ")            # a single space is a non-empty string → truthy
# True

1000 is 1000        # 'is' tests SAME OBJECT, not equal value, do not use it for
# may be True or False depending on Python, use == instead
```

The dangerous ones are silent. `bool("False")` is `True` because it is a non-empty string, so checking user input by truthiness alone can accept the literal word "False". And `is` compares identity, not value; use `==` for values and reserve `is` for `None`.

? THINK ABOUT IT

A program reads a setting as text and does `if user_setting:` to decide whether it is "on". The user types the word `False`. What happens, and why is it a bug? How would you check it correctly?

▸ Reveal the thinking

REFERENCE Quick reference

Operator	Meaning
`== !=`	equal, not equal
`< > <= >=`	ordering
`and or not`	combine / negate
`in` · `not in`	membership
`is` · `is not`	same object? (use only for `None`)

Falsy values (everything else is truthy): `False` · `None` · `0` · `0.0` · `""` · `[]` · `()` · `{}` · `set()`

TIP Write `if not items:` rather than `if len(items) == 0:`. It reads better and works for any container, thanks to truthiness.

✎ EXERCISES

1. Write a condition that is true when `n` is between 1 and 100 inclusive.
2. Write a "is a teenager" condition for ages 13 to 19.
3. Use `or` to give `nickname` a default of `"Anonymous"` when empty.
4. **Edge case:** why is `bool("0")` equal to `True` even though `bool(0)` is `False`?
5. **Think:** predict each: `bool([])`, `bool([[]])`, `bool(" ")`, `bool(0.0)`. Explain the pattern.

▸ Show answers

MODULE 6 · BEGINNER · STAGE 3

Lists

The workhorse collection: ordered, changeable, sliceable, and the place where the label-not-a-box idea finally bites.

Needs: Modules 2, 4 **Teaches:** mutation · slicing · comprehensions · aliasing

Then you can: store and process many values

CONCEPT Ordered and mutable

So far each name has held a single value. A **list** holds many values in order under one name. It is the workhorse of Python: a to-do list, the lines of a file, the rows of a table all

live naturally in lists. You write one in square brackets, and it may hold any types, even mixed.

Two properties define a list. It is **ordered**, so every item has a position you reach by index, and slicing pulls out ranges exactly as with strings. And it is **mutable**, meaning you can change it in place after creation: add, remove, replace, sort. That is the sharp difference from strings, which are frozen once made. Because building a new list from an old one is so common, Python offers a one-line form called a **list comprehension** you will use constantly.

DEFINITION

List

An ordered, mutable sequence of values, written in square brackets `[1, 2, 3]`. Ordered means items keep their position and are reached by index; mutable means the list can be changed in place after it is made.

EXAMPLES Code you can run

Example 6.1 · Changing a list in place

PYTHON

```python
tasks = ["email", "lunch"]
tasks.append("call")     # add one item to the END
tasks.insert(0, "wake")  # add at a given position (the front here)
tasks.remove("lunch")    # delete the first item equal to "lunch"
print(tasks)              # the SAME list object, now changed
```

OUTPUT

```
['wake', 'email', 'call']
```

Each method changes the existing list; none returns a new one. That is what "mutable" means, and it is why you do not write `tasks = tasks.append(...)`.

Example 6.2 · The list comprehension

PYTHON

```python
nums = [1, 2, 3, 4, 5, 6]
squares = [n * n for n in nums]           # transform EVERY item
evens   = [n for n in nums if n % 2 == 0]  # KEEP only some items (add an if)
print(squares)
print(evens)
```

OUTPUT

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
```

Read it as a sentence: "n times n, for each n in nums". The optional `if` filters. This single line replaces a four-line loop that appends to a new list.

EDGE CASES The errors, and the famous aliasing bug

⚠ EDGE CASES & COMMON ERRORS

PYTHON

```python
xs = [1, 2, 3]
xs[5]                  # no index 5 exists
# IndexError: list index out of range

xs.remove(99)          # 99 is not in the list
# ValueError: list.remove(x): x not in list

# The aliasing bug: = does NOT copy a list
a = [1, 2, 3]
b = a                  # b is the SAME list, not a copy (two labels, one object)
b.append(4)
print(a)               # [1, 2, 3, 4], changing b changed a!
# [1, 2, 3, 4]
```

The first two raise loud errors. The third is the silent classic, and it is exactly the label-not-a-box idea from Module 2: `b = a` ties a second label to one list. Mutate through either name and both see it. To get an independent copy, ask for one: `b = a.copy()` (or `a[:]`, or `list(a)`).

? THINK ABOUT IT

You want a 3-by-3 grid and write `grid = [[0] * 3] * 3`. You set `grid[0][0] = 9` and find that *three* cells changed, not one. Before revealing, work out why, connecting it to the aliasing bug above.

▸ Reveal the thinking

REFERENCE Quick reference

Method	Does
`append(x)`	add one item to the end
`insert(i, x)`	insert at position i
`extend(seq)`	add all items from another sequence
`remove(x)`	delete the first item equal to x
`pop([i])`	remove and return last (or item i)
`sort()` · `sorted(xs)`	sort in place · return a new sorted list
`reverse()`	reverse in place
`a.copy()` · `a[:]`	make an independent copy

TIP `sort()` changes the list and returns `None`; `sorted(xs)` leaves it alone and returns a new list. Writing `xs = xs.sort()` sets `xs` to `None`, a frequent slip.

✎ EXERCISES

1. Make a list of three colours, add a fourth, then remove the first.
2. Get the last two items of `[1,2,3,4,5]` with a slice.
3. Build `[1, 4, 9, 16, 25]` with a comprehension.
4. From `[5,12,8,3,20,7]`, keep only numbers greater than 8.
5. **Edge case:** what does `[1,2,3][5]` raise, and what does `[].pop()` raise?
6. **Think:** demonstrate the aliasing bug with two names on one list, then fix it so changing one leaves the other untouched.

▸ **Show answers**